

PROJECT_CONTEXT.md

Proyecto

Backend de autenticación construido con:

- Node.js
- Express
- SQLite
- JWT
- bcrypt

Objetivo:

Crear un sistema backend profesional de autenticación sin frontend usando Postman para pruebas.

Stack Tecnológico

Backend

- Node.js
- Express.js

Base de datos

- SQLite3

Seguridad

- bcrypt
- jsonwebtoken
- helmet
- cors

Variables de entorno

- dotenv

Desarrollo

- nodemon
-

Arquitectura del Proyecto

```
backend/
├── docs/
│   └── PROJECT_CONTEXT.md
├── node_modules/
├── src/
│   ├── config/
│   │   └── env.js
│   ├── controllers/
│   │   └── auth.controller.js
│   ├── database/
│   │   ├── db.js
│   │   └── init.js
│   ├── middlewares/
│   │   └── auth.middleware.js
│   ├── models/
│   │   └── user.model.js
│   ├── routes/
│   │   └── auth.routes.js
│   ├── utils/
│   └── server.js
├── .env
├── .gitignore
├── database.sqlite
├── package.json
└── package-lock.json
```

Arquitectura Backend

Patrón utilizado:

```
Routes → Controllers → Models → SQLite
```

Responsabilidades

Routes

Definen endpoints HTTP.

Ejemplo:

```
POST /api/auth/login
```

Controllers

Gestionan lógica de negocio:

- validaciones
 - respuestas
 - generación JWT
 - bcrypt
 - errores
-

Models

Gestionan acceso a SQLite:

- SELECT
 - INSERT
 - UPDATE
 - DELETE
-

Middlewares

Procesan peticiones antes del controlador:

- validación JWT
 - autenticación
-

Variables de Entorno

Archivo:

```
.env
```

Contenido:

```
PORT=3000
JWT_SECRET=CLAVE_SECRETA_LARGA
```

Configuración Actual

Express Middleware

```
app.use(helmet());
app.use(cors());
app.use(express.json());
```

Base de Datos

Motor:

```
SQLite3
```

Archivo físico:

```
database.sqlite
```

Tabla users

```
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password TEXT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Flujo de Registro

Endpoint:

```
POST /api/auth/register
```

Proceso

1. Validar datos recibidos
2. Verificar email existente
3. Hash password con bcrypt
4. Guardar usuario
5. Devolver respuesta JSON

Flujo de Login

Endpoint:

```
POST /api/auth/login
```

Proceso

1. Buscar usuario por email
2. Comparar password con bcrypt.compare()
3. Generar JWT
4. Devolver token

JWT

Biblioteca:

```
jsonwebtoken
```

Payload actual

```
{
  "id": 1,
  "email": "user@test.com"
}
```

Expiración JWT

```
expiresIn: '1h'
```

Propiedades JWT

iat

Issued At:

fecha de creación del token.

exp

Expiration:

fecha de expiración del token.

Middleware de Autenticación

Archivo:

```
middlewares/auth.middleware.js
```

Responsabilidades

- Leer Authorization Header
- Extraer Bearer Token
- Verificar JWT
- Bloquear accesos inválidos
- Añadir usuario a req.user

Rutas Actuales

Registro

```
POST /api/auth/register
```

Body:

```
{
  "name": "Marc",
  "email": "marc@test.com",
  "password": "123456"
}
```

Login

POST /api/auth/login

Body:

```
{
  "email": "marc@test.com",
  "password": "123456"
}
```

Ruta protegida

GET /api/auth/profile

Headers:

Authorization: Bearer TOKEN

Seguridad Implementada

bcrypt

Las contraseñas nunca se almacenan en texto plano.

helmet

Protección básica HTTP headers.

CORS

Control acceso entre dominios.

JWT

Autenticación stateless.

Passwords

Nunca se devuelven al cliente.

Testing

Herramienta usada:

Postman

No existe frontend actualmente.

Decisiones Arquitectónicas

SQLite

Usado por simplicidad y portabilidad.

Ideal para:

- aprendizaje
 - prototipos
 - pequeños proyectos
-

MVC Ligero

Separación clara:

- routes
 - controllers
 - models
-

Stateless Authentication

JWT evita sesiones tradicionales.

Próximas Mejoras Planeadas

Seguridad

- rate limiting
 - refresh tokens
 - validación avanzada
 - sanitización
 - blacklist tokens
-

Usuarios

- roles
 - permisos
 - recuperación password
 - verificación email
-

Backend

- logger
 - manejo centralizado errores
 - variables entorno avanzadas
 - tests automáticos
-

Objetivo del Proyecto

Crear una plantilla profesional reutilizable para futuros proyectos backend Node.js con autenticación segura.